Day 11/21: Python Comprehensions & Lambdas

The Data Refinery Sprint | Author: Prasad S Hiremath | Topic: Functional Transformations & Clean Code

✨ **Dasara Greetings & Daily Motivation (Vijayadashami Special)**
Wishing everyone in **The Data Refinery** community a very Happy and Auspicious Dasara! As we celebrate Vijayadashami—the triumph of knowledge over ignorance—let us channel the inner strength and wisdom of Goddess Durga.

> ***"Ya Devi Sarvabhuteshu Buddhi-Rupeena Samsthita, Namastasyai Namastasyai Namastasyai Namo Namah."***

May this festival empower your intellect, eliminate every bug in your code, and guide you to ultimate victory in your data engineering journey! Let's conquer Day 11! 🏹⚡

🏢 Real-World Case Study: The Bulky Data Cleaning Script

At **InstaDrop**, an ingestion pipeline parses transaction amounts from incoming streams. A junior engineer wrote a verbose loop to cast raw text values into positive floats, clogging the script with manual boilerplate and inline error blocks. In production, senior engineers decouple validation from iteration using **Comprehensions** paired with single-responsibility helper functions.

💡 Senior vs. Junior Pattern Comparison

❌ **Junior Pattern (Verbose Loop)**

```
raw = ["100.5", "-20", "invalid"] clean = [] for amt in raw: try: val = float(amt) if
```

📱 Quick Reference: Python Comprehension Types

Type	Syntax Template	Data Engineering Use Case
List Comp	`[expr for item in iter if cond]`	Filtering and transforming raw stream records inline.
Dict Comp	`{k_expr: v_expr for item in iter}`	Mapping primary keys to record metadata or lookup tables.
Set Comp	`{expr for item in iter}`	Instantly deduplicating categories or lookup tokens.
Lambda	`lambda args: expression`	Inline key extraction in `sorted()`, PySpark, or Pandas.

💻 Medium-Level Coding Challenge: Price Normalizer

Task: Write `normalize_prices(catalog)` to strip `"$"` symbols, cast prices to `float`, filter items under `$10.00`, and return qualifying product names.

```
def normalize_prices(catalog): """ Normalizes string prices and filters items under $10 u
```

❓ Part 1: Core Practice MCQs

Q1: Which list comprehension correctly extracts even numbers from 0 to 9?

A) [x for x in range(10) if x % 2 == 0] | B) (x for x in range(10) where x % 2 == 0)
C) list(range(10).filter(lambda x: x % 2 == 0)) | D) {x for x in range(10) if x % 2 == 0}

Key: A — Square brackets [] define a list comprehension with an optional trailing 'if' filter.

Q2: What is a key restriction of lambda functions in Python?

A) Cannot accept arguments | B) Restricted to a single expression and no multi-line control blocks
C) Must return string values | D) Cannot be used inside list comprehensions

Key: B — Lambdas evaluate a single expression implicitly and prohibit multi-line statements.

❓ Core Practice MCQs (Continued)

Q3: What is the output of `res = {x: x**2 for x in range(3)}`?

A) [0, 1, 4] | B) {0: 0, 1: 1, 2: 4} | C) {(0, 0), (1, 1), (2, 4)} | D) {'0': 0, '1': 1, '2': 4}

Key: B — The key:value colon syntax inside curly braces builds a dictionary comprehension.

Q4: Why are list comprehensions generally preferred in Python data pipelines?

A) Optimized at C-bytecode level, making simple transformations cleaner and slightly faster
B) Automatically encrypt records | C) Eliminate RAM usage | D) Convert lists to SQL tables

Key: A — Comprehensions avoid Python function call overhead of .append() in standard loops.

Q5: What is the output length of `{x * x for x in [1, 2, 2, 3, 3, 3]}`?

A) 6 | B) 5 | C) 3 | D) 1

Key: C — Set comprehensions evaluate to unique values, deduplicating output to {1, 4, 9}.

💡 Part 2: 5 Bonus Technical MCQs (Interview Focus)

BQ1: What type of object is returned by `(x * 2 for x in range(1000000))`?

A) List | B) Set | C) Generator Expression | D) Tuple Comprehension

Key: C — Parentheses () define a lazy generator expression, evaluating items on demand.

BQ2: How do you sort a list of dicts `records` by `"timestamp"` ascending using lambda?

A) sorted(records, key=lambda x: x['timestamp']) | B) records.sort(by='timestamp')
C) sort(records, lambda: 'timestamp') | D) sorted(records.timestamp)

Key: A — The 'key' parameter accepts a lambda function returning the dictionary value to sort by.

BQ3: What is the value of `[x if x > 0 else 0 for x in [-2, 5, -1, 3]]`?

A) [5, 3] | B) [0, 5, 0, 3] | C) [-2, 5, -1, 3] | D) Syntax Error

Key: B — A ternary if-else placement before 'for' transforms values; an 'if' after 'for' filters elements.

BQ4: What happens if you modify a outer scope variable directly inside a lambda expression?

A) Raises SyntaxError | B) Reads the variable's value dynamically at call time (late binding)
C) Instantly copies the variable value | D) Converts variable to global

Key: B — Lambdas look up variables in enclosing scope when called, adhering to late binding rules.

BQ5: Which built-in function pairs most cleanly with lambda for element-wise filtering?

A) map() | B) filter() | C) reduce() | D) zip()

Key: B — filter(lambda_func, iterable) returns items where the boolean lambda evaluates to True.